

Topological Sort + DAG DP — Complete Revision Notes

Study Context

This document summarizes the Topological Sort / DAG DP problems and doubts discussed in this chat, in the order they were studied.

1. Course Schedule — LeetCode 207

Problem Link

[Open on Platform](#)

Problem Statement

You are given n courses and prerequisite pairs. Determine whether all courses can be completed. A prerequisite $u \rightarrow v$ means u must be completed before v .

Doubt I Asked

- Started the Topological Sort practice sequence with Course Schedule.
- Later asked to move through Topological Sort problems in solving order.

Approach

Use Kahn's Algorithm:

1. Build the directed graph.
2. Compute indegree of every node.
3. Put all nodes with indegree 0 into a queue.
4. Remove them one by one and reduce the indegree of their neighbours.
5. If all n nodes are processed, there is no cycle; otherwise a cycle exists.

DP State / Graph State / Data Structure State

- `indegree[u]`: number of prerequisites still remaining for u .
- Queue: currently available courses.

Transition / Logic

For every edge $u \rightarrow v$:

```
indegree[v]--;  
if (indegree[v] == 0)  
    q.push(v);
```

C++ Code

```
class Solution {
public:
    bool canFinish(int n, vector<vector<int>>& prerequisites) {
        vector<vector<int>> adj(n);
        vector<int> indegree(n, 0);

        for (auto &e : prerequisites) {
            int u = e[1];
            int v = e[0];

            adj[u].push_back(v);
            indegree[v]++;
        }

        queue<int> q;

        for (int i = 0; i < n; i++)
            if (indegree[i] == 0)
                q.push(i);

        int processed = 0;

        while (!q.empty()) {
            int u = q.front();
            q.pop();
            processed++;

            for (int v : adj[u]) {
                indegree[v]--;

                if (indegree[v] == 0)
                    q.push(v);
            }
        }

        return processed == n;
    }
};
```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why does `processed != n` mean a cycle exists?
- Why do we start with indegree-zero nodes?
- Why can Kahn's algorithm detect cycles?

What I Learned

Kahn's Algorithm can be used both for generating a topological order and for detecting a cycle in a directed graph.

2. Course Schedule II — LeetCode 210

Problem Link

[Open on Platform](#)

Problem Statement

Return one valid order in which all courses can be completed. If impossible because of a cycle, return an empty array.

Doubt I Asked

- The direct problem link initially opened the previous problem.
- The key distinction was that Course Schedule II asks for the actual topological ordering.

Approach

Use the same Kahn's Algorithm as Course Schedule, but store every popped node in `topo`.

DP State / Graph State / Data Structure State

- `indegree`
- `queue`
- `topo`: the resulting topological order.

Transition / Logic

Whenever a node becomes indegree zero, push it into the queue. Every popped node is appended to `topo`.

C++ Code

```
class Solution {
public:
    vector<int> findOrder(int n, vector<vector<int>>& prerequisites) {
        vector<vector<int>> adj(n);
        vector<int> indegree(n, 0);

        for (auto &e : prerequisites) {
            int u = e[1];
            int v = e[0];

            adj[u].push_back(v);
            indegree[v]++;
        }

        queue<int> q;
```

```

    for (int i = 0; i < n; i++)
        if (indegree[i] == 0)
            q.push(i);

    vector<int> topo;

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        topo.push_back(u);

        for (int v : adj[u]) {
            indegree[v]--;

            if (indegree[v] == 0)
                q.push(v);
        }

        if ((int)topo.size() != n)
            return {};

        return topo;
    }
};

```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Difference between Course Schedule and Course Schedule II?
- Why does storing popped nodes give a topological order?

What I Learned

A small change to Kahn's Algorithm converts cycle detection into actual topological-order construction.

3. Course Schedule — CSES

Problem Link

[Open on Platform](#)

Problem Statement

Determine whether all courses can be ordered while respecting prerequisite relationships.

Doubt I Asked

- Used the problem as another Topological Sort practice problem.

Approach

Kahn's Algorithm.

DP State / Graph State / Data Structure State

- indegree
- queue
- topo

Transition / Logic

Process all indegree-zero nodes and remove their outgoing edges.

C++ Code

```
// Same Kahn's Algorithm pattern used above.
// The important revision template is:

queue<int> q;

for (int i = 1; i <= n; i++)
    if (indegree[i] == 0)
        q.push(i);

while (!q.empty()) {
    int u = q.front();
    q.pop();

    topo.push_back(u);

    for (int v : adj[u]) {
        indegree[v]--;

        if (indegree[v] == 0)
            q.push(v);
    }
}
```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- How is Kahn different from DFS-based topological sorting?
- When should Topological Sort be preferred?

What I Learned

Kahn's Algorithm is a reusable template for DAG ordering.

4. Find All Possible Recipes from Given Supplies — LeetCode 2115

Problem Link

[Open on Platform](#)

Problem Statement

Recipes depend on ingredients, and some ingredients may themselves be recipes. Determine all recipes that can eventually be made from the available supplies.

Doubt I Asked

- Included as a Topological Sort problem in the solving sequence.

Approach

Model the dependency graph as a DAG-style dependency problem:

- Ingredients point to recipes that need them.
- A recipe becomes available when all its required ingredients have been satisfied.
- Use indegree to represent the number of unsatisfied ingredients.

DP State / Graph State / Data Structure State

- `indegree[recipe]`: remaining ingredients.
- Queue: currently available supplies/recipes.

Transition / Logic

When an available ingredient is processed, reduce the indegree of every recipe depending on it.

C++ Code

```
class Solution {
public:
    vector<string> findAllRecipes(
        vector<string>& recipes,
        vector<vector<string>>& ingredients,
        vector<string>& supplies
    ) {
        unordered_map<string, vector<string>> adj;
        unordered_map<string, int> indegree;

        for (int i = 0; i < (int)recipes.size(); i++) {
            indegree[recipes[i]] = ingredients[i].size();
        }
    }
};
```

```

        for (auto &x : ingredients[i])
            adj[x].push_back(recipes[i]);
    }

    queue<string> q;

    for (auto &s : supplies)
        q.push(s);

    vector<string> ans;

    while (!q.empty()) {
        string item = q.front();
        q.pop();

        for (auto &recipe : adj[item]) {
            indegree[recipe]--;

            if (indegree[recipe] == 0) {
                ans.push_back(recipe);
                q.push(recipe);
            }
        }
    }

    return ans;
}
};

```

Time Complexity

$O(\text{total number of ingredient dependencies})$

Space Complexity

$O(\text{total number of dependencies})$

Possible Doubts During Revision

- Why can supplies be treated as initially available nodes?
- Why does indegree represent missing ingredients?

What I Learned

Topological sorting is not limited to ordinary graphs; it also models dependency systems.

5. Parallel Courses — LeetCode 1136

Problem Link

[Open on Platform](#)

Problem Statement

Courses have prerequisite relations. In one semester, any number of currently available courses can be taken. Find the minimum number of semesters needed, or **-1** if there is a cycle.

Doubt I Asked

- Asked to write the complete problem with test cases.
- Wrote a Kahn implementation using levels/semesters.

Approach

Kahn's Algorithm with BFS levels:

- All currently indegree-zero nodes can be taken in the same semester.
- Process the current queue size as one semester.
- Count semesters.

DP State / Graph State / Data Structure State

- **indegree**
- queue
- **semester**
- **topo**

Transition / Logic

At the beginning of every semester:

```
int k = q.size();
```

Process exactly those currently available courses.

C++ Code

```
pair<int, vector<int>> kahns_algorithm(vector<vector<int>> &adj) {
    int n = adj.size() - 1;

    vector<int> indegree(n + 1);

    for (int u = 1; u <= n; u++)
        for (int v : adj[u])
            indegree[v]++;

    queue<int> q;

    for (int u = 1; u <= n; u++)
        if (indegree[u] == 0)
            q.push(u);

    vector<int> topo;
    int semester = 0;
```



```

while (!q.empty()) {
    semester++;

    int sz = q.size();

    while (sz-- > 0) {
        int u = q.front();
        q.pop();

        topo.push_back(u);

        for (int v : adj[u]) {
            indegree[v]--;

            if (indegree[v] == 0)
                q.push(v);
        }
    }

    return {semester, topo};
}

```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why does one queue level correspond to one semester?
- Why are newly unlocked nodes taken in the next semester?

What I Learned

BFS levels on Kahn's Algorithm can represent rounds, semesters, or phases.

6. Parallel Courses III — LeetCode 2050

Problem Link

[Open on Platform](#)

Problem Statement

Each course has a duration. A course can start only after all prerequisites are completed. Find the minimum total time to complete all courses.

Doubt I Asked

- Asked whether this can be solved using DP on DAG after first doing Topological Sort.

- Asked to understand: **Topo Sort -> Process nodes in topo order -> $dp[v] = \max(dp[v], dp[u] + \text{weight}(v))$.**
- Asked to first extract Topological Sort and then run DP separately.

Approach

First build the topological order. Then perform forward DP.

For each course:

- **$dp[u]$ = earliest finish time of course u .**
- For a source course, **$dp[u] = \text{time}[u]$.**
- For edge **$u \rightarrow v$** , course **v** can finish only after **u** finishes and **v** itself takes **$\text{time}[v]$.**

DP State / Graph State / Data Structure State

$dp[u]$ = earliest time at which course u can be completed.

Transition / Logic

$dp[v] = \max(dp[v], dp[u] + \text{time}[v]);$

Why **max**? **v** must wait for its slowest/last prerequisite to finish.

C++ Code

```
class Solution {
public:
    int minimumTime(int n, vector<vector<int>>& relations, vector<int>& time) {

        vector<vector<int>> adj(n + 1);
        vector<int> finish(n + 1, 0);
        vector<int> indegree(n + 1, 0);

        for (auto &e : relations) {
            int u = e[0];
            int v = e[1];

            adj[u].push_back(v);
            indegree[v]++;
        }

        queue<int> q;

        for (int i = 1; i <= n; i++) {
            if (indegree[i] == 0) {
                q.push(i);
                finish[i] = time[i - 1];
            }
        }

        while (!q.empty()) {
            int u = q.front();
```

```

        q.pop();

        for (auto nei : adj[u]) {
            finish[nei] = max(finish[nei],
                             finish[u] + time[nei - 1]);

            indegree[nei]--;

            if (indegree[nei] == 0)
                q.push(nei);
        }

        int ans = 0;

        for (int i = 1; i <= n; i++)
            ans = max(ans, finish[i]);

        return ans;
    }
};

```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why is **max** used instead of **min**?
- Why do we add **time[v]**, not **time[u]**?
- Why can all courses with no dependency be started together?

What I Learned

This was the first strong **forward DP on DAG** pattern:

```

Topo Sort
↓
process u
↓
for every u -> v
↓
dp[v] = max(dp[v], dp[u] + contribution_of_v)

```

7. AtCoder DP G — Longest Path

Problem Link

[Open on Platform](#)

Problem Statement

Given a DAG, find the length of the longest path. Length is the number of edges.

Doubt I Asked

- Asked to explain the problem.
- Asked why the DP can be done using Topological Sort.
- Asked whether the forward DP implementation is also correct.
- Confirmed that the user's code was correct.

Approach

The user's implementation uses forward DP:

- First obtain a topological order.
- `longest[u]` = longest path ending at `u`.
- Process nodes in topological order.
- Propagate the value to outgoing neighbours.

DP State / Graph State / Data Structure State

`longest[u]` = longest path ending at node `u`.

Transition / Logic

`longest[v] = max(longest[v], longest[u] + 1);`

C++ Code

```
int32_t main() {  
  
    int n, m;  
    cin >> n >> m;  
  
    vector<vector<int>> adj(n + 1);  
    vector<int> indegree(n + 1, 0);  
  
    for (int i = 0; i < m; i++) {  
        int u, v;  
        cin >> u >> v;  
  
        adj[u].push_back(v);  
        indegree[v]++;  
    }  
}
```

```

vector<int> longest(n + 1, 0);

queue<int> q;

for (int i = 1; i <= n; i++) {
    if (indegree[i] == 0)
        q.push(i);
}

while (!q.empty()) {
    int u = q.front();
    q.pop();

    for (int v : adj[u]) {
        longest[v] = max(longest[v], longest[u] + 1);

        indegree[v]--;

        if (indegree[v] == 0)
            q.push(v);
    }
}

int ans = 0;

for (int i = 1; i <= n; i++)
    ans = max(ans, longest[i]);

cout << ans << '\n';
}

```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Difference between $dp[u]$ = longest path ending at u and $dp[u]$ = longest path starting from u .
- Why can forward DP use normal topological order?
- When should reverse topological order be used?

What I Learned

There are two equivalent longest-path DP definitions:

Forward:

```
dp[v] = max(dp[v], dp[u] + 1);
```

Process normal topo order.

Reverse:

```
dp[u] = max(dp[u], dp[v] + 1);
```

Process reverse topo order.

The processing order is determined by the DP dependency.

8. CSES — Longest Flight Route

Problem Link

[Open on Platform](#)

Problem Statement

Find the longest route from node **1** to node **n** in a DAG and print the route itself. If no route exists, print **IMPOSSIBLE**.

Doubt I Asked

- Initially wrote a reverse-DP solution using **longest[u]** and **child[u]**.
- Asked whether that logic was wrong.
- Learned that the reverse DP itself is valid for "longest path anywhere", but the CSES problem specifically requires a path from **1** to **n**.
- Asked why reverse topological order is needed if **dp[u]** depends on **dp[v]**.
- Asked about:

```
for (int i = topo.size()-1; i >= 0; i--)
```

Approach

Use:

1. Kahn's Algorithm to get a topological order.
2. Forward DP from node **1**.
3. **dp[u]** = longest path from **1** to **u**.
4. Store **parent[v]** whenever **dp[v]** improves.
5. Reconstruct the path from **n** back to **1**.

DP State / Graph State / Data Structure State

```
dp[u] = maximum number of nodes in a path from 1 to u.
```

Use **-1** for unreachable nodes.

```
dp[1] = 1;
```

Transition / Logic

```
dp[v] = max(dp[v], dp[u] + 1);  
parent[v] = u;
```

Only process **u** if it is reachable from **1**.

C++ Code

```
#include <bits/stdc++.h>  
using namespace std;  
  
#define int long long  
  
int32_t main() {  
  
    int n, m;  
    cin >> n >> m;  
  
    vector<vector<int>> adj(n + 1);  
    vector<int> indegree(n + 1, 0);  
  
    for (int i = 0; i < m; i++) {  
        int u, v;  
        cin >> u >> v;  
  
        adj[u].push_back(v);  
        indegree[v]++;  
    }  
  
    // ----- Topological Sort -----  
    queue<int> q;  
    vector<int> topo;  
  
    for (int i = 1; i <= n; i++) {  
        if (indegree[i] == 0)  
            q.push(i);  
    }  
  
    while (!q.empty()) {  
        int u = q.front();  
        q.pop();  
  
        topo.push_back(u);  
  
        for (int v : adj[u]) {  
            indegree[v]--;  
  
            if (indegree[v] == 0)  
                q.push(v);  
        }  
    }  
  
    // ----- Forward DP -----  
    vector<int> dp(n + 1, -1);  
    vector<int> parent(n + 1, -1);
```

```

// Source node: path contains node 1 itself
dp[1] = 1;

for (int u : topo) {

    // Ignore nodes that cannot be reached from node 1
    if (dp[u] == -1)
        continue;

    for (int v : adj[u]) {

        // Extend the path from u to v
        if (dp[v] < dp[u] + 1) {
            dp[v] = dp[u] + 1;
            parent[v] = u;
        }
    }
}

if (dp[n] == -1) {
    cout << "IMPOSSIBLE\n";
    return 0;
}

// ----- Path Reconstruction -----
vector<int> path;

int cur = n;

while (cur != -1) {
    path.push_back(cur);
    cur = parent[cur];
}

reverse(path.begin(), path.end());

cout << path.size() << '\n';

for (int x : path)
    cout << x << ' ';

cout << '\n';
}

```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why is $dp[1] = 1$ and not 0?

- Why use `-1` for all other nodes?
- Why does `parent[v] = u` reconstruct the path?
- Why is `dp[u] = longest path from u to n` a different state?
- Why does reverse DP require reverse topo?

What I Learned

A crucial distinction:

AtCoder DP G:

Longest path anywhere in the DAG.

CSES Longest Flight Route:

Longest path specifically from 1 to n.

Therefore CSES needs source-aware DP and path reconstruction.

9. Reverse Topological DP Concept

Doubt I Asked

- "why reverse topo"
- Asked about:

```
for (int i = topo.size()-1; i >= 0; i--)
```

Approach

Do not memorize "reverse topo". Instead inspect the recurrence.

If:

```
dp[v] = f(dp[u]);
```

then `dp[v]` depends on the parent `u`, so process:

Topological Order

If:

```
dp[u] = f(dp[v]);
```

then `dp[u]` depends on its child `v`, so process:

Reverse Topological Order

Key Example

For:

```
1 -> 2 -> 3
```

Forward:

```
dp[1] -> dp[2] -> dp[3]
```

Use:

```
1 2 3
```

Reverse:

```
dp[3] -> dp[2] -> dp[1]
```

Use:

```
3 2 1
```

C++ Pattern

```
for (int i = (int)topo.size() - 1; i >= 0; i--) {  
    int u = topo[i];  
  
    // dp[u] depends on dp[v]  
}
```

What I Learned

The DP dependency decides the traversal order.

10. CSES — Game Routes

Problem Link

[Open on Platform](#)

Problem Statement

Count the number of different routes from node **1** to node **n** in a DAG, modulo **1e9+7**.

Doubt I Asked

- Asked for the problem.
- Explained that the problem can be solved with reverse DP.
- The user clarified that they solved it using:

```
dp[n] = 0;  
for(auto it : adj[u])  
    dp[u] += dp[v];
```

Approach

Use reverse DP.

The correct base case is:

```
dp[n] = 1;
```

Then process nodes in reverse topological order.

DP State / Graph State / Data Structure State

$dp[u]$ = number of paths from u to n .

Transition / Logic

$dp[u] += dp[v];$

for every edge:

$u \rightarrow v$

because every path from v to n becomes a path from u to n by adding edge $u \rightarrow v$.

C++ Code

```
// Reverse-topological DP pattern

vector<int> dp(n + 1, 0);

// There is exactly one empty continuation at the destination.
dp[n] = 1;

for (int i = (int)topo.size() - 1; i >= 0; i--) {
    int u = topo[i];
    for (int v : adj[u]) {
        dp[u] += dp[v];
        if (dp[u] >= MOD)
            dp[u] -= MOD;
    }
}

// Number of routes from 1 to n
cout << dp[1] << '\n';
```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why is $dp[n] = 1$, not 0 ?
- Why reverse topo?
- Why is the operation sum instead of max ?

What I Learned

The same DAG DP structure can count paths instead of maximizing a value:

```
dp[u] += dp[v];
```

11. All Ancestors of a Node in a DAG — LeetCode 2192

Problem Link

[Open on Platform](#)

Problem Statement

For every node, return all nodes that can reach it.

Doubt I Asked

- Wrote a Kahn's Algorithm solution that propagated ancestor vectors.
- Asked about MLE.
- Then asked how to use a **set** to avoid duplicates.
- Asked how to convert an already-defined map/set container into a vector.
- Clarified that the vector was already defined.

Approach

Topological propagation:

- **ancestor[u]** stores all ancestors known for **u**.
- For edge **u -> v**, add **u** and all ancestors of **u** into **ancestor[v]**.
- Since the same ancestor can arrive through multiple parents, duplicates must be removed.

DP State / Graph State / Data Structure State

ancestor[u] = set of all ancestors of **u**.

Transition / Logic

```
ancestor[v].insert(u);  
  
for (int x : ancestor[u])  
    ancestor[v].insert(x);
```

Using **set** prevents duplicate ancestors from being stored repeatedly.

C++ Code

```
class Solution {  
    vector<vector<int>> kahns_algorithm(vector<vector<int>> &adj) {  
  
        int n = adj.size();  
  
        vector<int> indegree(n, 0);
```

```

    for (int u = 0; u < n; u++)
        for (int v : adj[u])
            indegree[v]++;

    queue<int> q;

    for (int u = 0; u < n; u++)
        if (indegree[u] == 0)
            q.push(u);

    // Set avoids duplicate ancestors.
    vector<set<int>> ancestor(n);

    while (!q.empty()) {

        int u = q.front();
        q.pop();

        for (int v : adj[u]) {

            // u itself is an ancestor of v.
            ancestor[v].insert(u);

            // Every ancestor of u is also an ancestor of v.
            for (int x : ancestor[u])
                ancestor[v].insert(x);

            indegree[v]--;

            if (indegree[v] == 0)
                q.push(v);
        }
    }

    // Convert each set to the already-defined vector.
    vector<vector<int>> ans(n);

    for (int i = 0; i < n; i++)
        ans[i].assign(ancestor[i].begin(), ancestor[i].end());

    return ans;
}

public:
    vector<vector<int>> getAncestors(
        int n,
        vector<vector<int>>& edges
    ) {
        vector<vector<int>> adj(n);

        for (auto &e : edges) {
            int u = e[0];
            int v = e[1];

            adj[u].push_back(v);

```

```

    }
    return kahns_algorithm(adj);
}
};

```

Time Complexity

The exact complexity depends on the total number of propagated ancestors. With `set`, insertion has logarithmic overhead.

Space Complexity

Potentially $O(V^2)$ for the stored ancestor relationships; `set` nodes also have significant per-element memory overhead.

Possible Doubts During Revision

- Why did the original vector solution cause MLE?
- Why does sorting + `unique()` only remove duplicates after already storing them?
- Why does `set` prevent duplicate storage?
- How do I convert a set into an already-defined vector?

What I Learned

For an already-defined vector:

```
ans.assign(st.begin(), st.end());
```

For a map:

```
vector<pair<int,int>> v(mp.begin(), mp.end());
```

For existing vectors, `assign()` is the cleanest overwrite operation.

Important Note

Although `set` avoids duplicates, it is not automatically a memory optimization. `std::set` has substantial node overhead. The original vector approach can use huge memory because duplicates are stored before deduplication; a bitset-based solution is more memory-efficient for this particular constraint.

12. Loud and Rich — LeetCode 851

Problem Link

[Open on Platform](#)

Problem Statement

For every person, find the quietest person among that person and everyone richer than that person.

Doubt I Asked

- Asked to explain the problem.
- Connected it with the DAG DP patterns already learned.

Approach

Treat:

```
richer u -> v
```

as a directed edge from richer person to poorer person.

Propagate the best candidate from parent to child.

DP State / Graph State / Data Structure State

```
ans[u] = index of the quietest known person among u and all richer ancestors of u.
```

Initially:

```
ans[i] = i;
```

Transition / Logic

For:

```
u -> v
```

Compare:

```
quiet[ans[u]]
```

with:

```
quiet[ans[v]]
```

If **ans[u]** is quieter:

```
ans[v] = ans[u];
```

C++ Code

```
class Solution {
public:
    vector<int> loudAndRich(
        vector<vector<int>>& richer,
        vector<int>& quiet
    ) {
        int n = quiet.size();

        vector<vector<int>> adj(n);
        vector<int> indegree(n, 0);

        for (auto &e : richer) {
            int u = e[0];
            int v = e[1];

            // u is richer than v.
```

```

        adj[u].push_back(v);
        indegree[v]++;
    }

    queue<int> q;

    for (int i = 0; i < n; i++)
        if (indegree[i] == 0)
            q.push(i);

    vector<int> ans(n);

    // Initially, each person is their own best candidate.
    for (int i = 0; i < n; i++)
        ans[i] = i;

    while (!q.empty()) {

        int u = q.front();
        q.pop();

        for (int v : adj[u]) {

            // Propagate the quieter candidate from u to v.
            if (quiet[ans[u]] < quiet[ans[v]])
                ans[v] = ans[u];

            indegree[v]--;

            if (indegree[v] == 0)
                q.push(v);
        }
    }

    return ans;
}
};

```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why store an index instead of the quiet value itself?
- Why initialize `ans[i] = i`?
- Why does information flow from richer to poorer?

What I Learned

A DAG DP state does not have to be a number such as distance. It can store the **best candidate node/index**.

13. Codeforces 909E — Coprocessor

Problem Link

[Open on Platform](#)

Problem Statement

Tasks form a dependency DAG and each task is assigned to either the main processor or coprocessor. Main-processor tasks can be performed normally; coprocessor tasks are grouped into coprocessor activations. Minimize the number of coprocessor activations.

Doubt I Asked

- Asked for a hint.
- Asked why all CPU/type-0 tasks should be processed first.
- Asked for implementation.

Approach

Modified Kahn's Algorithm with two queues:

- `cpu`: type 0
- `cop`: type 1

Greedy rule:

1. Process all currently available CPU tasks first.
2. This is free and may unlock more CPU or coprocessor tasks.
3. When no CPU task is available and coprocessor tasks exist, start one coprocessor activation.
4. Process all coprocessor tasks that become available during that same activation.

DP State / Graph State / Data Structure State

No numerical DP is required.

State is:

- `indegree`
- `cpu` queue
- `cop` queue
- `ans` = number of coprocessor activations.

Transition / Logic

For every processed node:

```
indegree[v]--;  
  
if (indegree[v] == 0) {  
    if (type[v] == 0)  
        cpu.push(v);  
    else  
        cop.push(v);  
}
```

Important: Do **not** process only the initial `cop.size()` tasks. Newly unlocked coprocessor tasks must be processed in the same activation.

C++ Code

```
#include <bits/stdc++.h>  
using namespace std;  
  
#define int long long  
  
int32_t main()  
{  
    int n, m;  
    cin >> n >> m;  
  
    vector<int> type(n);  
  
    for (int i = 0; i < n; i++)  
        cin >> type[i];  
  
    vector<vector<int>> adj(n);  
    vector<int> indegree(n);  
  
    for (int i = 0; i < m; i++)  
    {  
        int u, v;  
        cin >> u >> v;  
  
        // u must be completed before v.  
        adj[u].push_back(v);  
        indegree[v]++;  
    }  
  
    queue<int> cpu;  
    queue<int> cop;  
  
    for (int i = 0; i < n; i++)  
    {  
        if (indegree[i] == 0)  
        {  
            if (type[i] == 0)  
                cpu.push(i);  
            else  
                cop.push(i);  
        }  
    }  
}
```

```

        cop.push(i);
    }
}

int ans = 0;

while (!cpu.empty() || !cop.empty())
{
    // CPU tasks are free, so process all of them first.
    while (!cpu.empty())
    {
        int u = cpu.front();
        cpu.pop();

        for (int v : adj[u])
        {
            indegree[v]--;

            if (indegree[v] == 0)
            {
                if (type[v] == 0)
                    cpu.push(v);
                else
                    cop.push(v);
            }
        }
    }

    if (cop.empty())
        continue;

    // Start one coprocessor activation.
    ans++;

    // Process every coprocessor task available during this activation.
    while (!cop.empty())
    {
        int u = cop.front();
        cop.pop();

        for (int v : adj[u])
        {
            indegree[v]--;

            if (indegree[v] == 0)
            {
                if (type[v] == 0)
                    cpu.push(v);
                else
                    cop.push(v);
            }
        }
    }
}

```

```
    cout << ans << '\n';  
}
```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why process all CPU tasks first?
- Why can newly unlocked coprocessor tasks be handled in the same activation?
- Why is this greedy approach better than arbitrarily choosing available tasks?

What I Learned

Think of:

```
CPU = free work  
COP = paid activation
```

So:

```
Process all CPU work  
    ↓  
Unlock maximum possible work  
    ↓  
Use one coprocessor activation  
    ↓  
Repeat
```

The core pattern is **Modified Kahn + Greedy queues**.

14. Codeforces 1572A — Book

Problem Link

[Open on Platform](#)

Problem Statement

There are n chapters with prerequisites. A chapter can be read after all its prerequisites. If a prerequisite has a smaller chapter number than the current chapter, it can be read in the same day; otherwise an additional day is needed. Find the minimum number of days, or -1 if the dependency graph contains a cycle.

Doubt I Asked

- Asked for an explanation.
- Asked for implementation.

Approach

Use Kahn's Algorithm plus a `day[]` DP.

For each edge:

`u -> v`

- If `u < v`, `v` can be read on the same day as `u`.
- If `u > v`, `v` needs the next day.

DP State / Graph State / Data Structure State

`day[u]` = earliest day on which chapter `u` can be read.

Initialize:

`day[i] = 1;`

Transition / Logic

```
if (u < v)
    day[v] = max(day[v], day[u]);
else
    day[v] = max(day[v], day[u] + 1);
```

C++ Code

```
#include <bits/stdc++.h>
using namespace std;

#define int long long

int32_t main()
{
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    int T;
    cin >> T;

    while (T--)
    {
        int n;
        cin >> n;

        vector<vector<int>> adj(n + 1);
        vector<int> indegree(n + 1);

        // Build dependency graph.
        for (int i = 1; i <= n; i++)
        {
            int k;
            cin >> k;

            while (k--)
            {
```

```

        int x;
        cin >> x;

        // x must be read before i.
        adj[x].push_back(i);
        indegree[i]++;
    }
}

queue<int> q;
vector<int> day(n + 1, 1);

for (int i = 1; i <= n; i++)
{
    if (indegree[i] == 0)
        q.push(i);
}

int processed = 0;

while (!q.empty())
{
    int u = q.front();
    q.pop();

    processed++;

    for (int v : adj[u])
    {
        // Same day if prerequisite index is smaller.
        if (u < v)
            day[v] = max(day[v], day[u]);
        else
            day[v] = max(day[v], day[u] + 1);

        indegree[v]--;

        if (indegree[v] == 0)
            q.push(v);
    }
}

// If not all chapters were processed, there is a cycle.
if (processed != n)
{
    cout << -1 << '\n';
    continue;
}

cout << *max_element(day.begin() + 1, day.end()) << '\n';
}
}

```

Time Complexity

$O(V + E)$

Space Complexity

$O(V + E)$

Possible Doubts During Revision

- Why does the transition depend on $u < v$?
- Why is this still DAG DP?
- Why does cycle detection remain the same as Kahn's Algorithm?

What I Learned

A DAG DP transition can depend on an extra property of the edge, such as node indices:

Topo Sort
↓
Edge-specific DP transition

Master Takeaways

1. Basic Kahn's Algorithm

```
queue<int> q;

for (int i = 0; i < n; i++)
    if (indegree[i] == 0)
        q.push(i);

while (!q.empty())
{
    int u = q.front();
    q.pop();

    for (int v : adj[u])
    {
        indegree[v]--;

        if (indegree[v] == 0)
            q.push(v);
    }
}
```

Use it for:

- Cycle detection
- Topological ordering
- Dependency scheduling

2. Topological Sort Can Be a DP Order

Core idea:

```
DAG
↓
Topological Order
↓
DP in dependency order
```

3. Forward DAG DP

If:

```
dp[v] depends on dp[u]
```

then process:

```
Topological Order
```

General template:

```
for (int u : topo)
{
    if (dp[u] is invalid)
        continue;

    for (int v : adj[u])
    {
        dp[v] = combine(dp[v], dp[u], v);
    }
}
```

Examples:

Longest path

```
dp[v] = max(dp[v], dp[u] + 1);
```

Parallel Courses III

```
finish[v] = max(finish[v], finish[u] + time[v]);
```

Loud and Rich

```
if (quiet[dp[u]] < quiet[dp[v]])
    dp[v] = dp[u];
```

4. Reverse DAG DP

If:

```
dp[u] depends on dp[v]
```


then process:

Reverse Topological Order

General template:

```
for (int i = (int)topo.size() - 1; i >= 0; i--)
{
    int u = topo[i];
    for (int v : adj[u])
    {
        dp[u] = combine(dp[u], dp[v]);
    }
}
```

Examples:

Longest path from u to n

```
dp[u] = max(dp[u], dp[v] + 1);
```

Game Routes

```
dp[u] += dp[v];
```

5. The Most Important Rule

Do not memorize:

```
"Always use normal topo"
"Always use reverse topo"
```

Instead ask:

My DP state depends on whose DP?

If:

```
parent's DP → child
```

use normal topo.

If:

```
child's DP → parent
```

use reverse topo.

6. DP Does Not Have to Store a Number

It can store:

Distance

```
dp[u] = longest distance
```

Time

```
dp[u] = finish time
```

Number of paths

```
dp[u] = number of ways
```

Best node

```
dp[u] = index of best candidate
```

Set of ancestors

```
ancestor[u] = set of ancestors
```

This is a major expansion of the meaning of "DP state".

7. Path Reconstruction

Whenever the problem asks for the actual path, store:

```
parent[v] = u;
```

Then:

```
vector<int> path;
int cur = target;
while (cur != -1)
{
    path.push_back(cur);
    cur = parent[cur];
}
reverse(path.begin(), path.end());
```

This was important in CSES Longest Flight Route.

8. Reachability Matters

For a fixed-source problem such as CSES Longest Flight Route:

```
vector<int> dp(n + 1, -1);
dp[1] = 1;
```

Do **not** initialize every node to 0, because that incorrectly treats unreachable nodes as reachable.

9. Kahn + Levels

For problems asking:

- minimum semesters

- minimum rounds
- number of phases

you can use:

```
int sz = q.size();
while (sz--)
{
    // one complete level / round
}
```

This was used for Parallel Courses.

10. Kahn + Greedy

Codeforces 909E showed that Topological Sort can be combined with greedy processing.

```
CPU tasks → free
COP tasks → costly activation
```

Therefore:

```
Process all CPU tasks
    ↓
Start one COP activation
    ↓
Process all COP tasks unlocked during it
    ↓
Repeat
```

11. Ancestor Propagation

LC 2192 introduced:

```
ancestor[v] = ancestor[v] ∪ {u} ∪ ancestor[u];
```

The important idea is:

In a DAG, once a parent has been completely processed, its accumulated information can be propagated to its children.

12. Container Conversion

If a vector is already defined:

```
vector<int> ans;
```

and you have a set:

```
set<int> st;
```

use:

```
ans.assign(st.begin(), st.end());
```

For a vector of vectors:

```
for (int i = 0; i < n; i++)  
    ans[i].assign(st[i].begin(), st[i].end());
```

For a map:

```
vector<pair<int,int>> v(mp.begin(), mp.end());
```

Final DAG Recognition Checklist

When you see a new problem, ask these questions in order:

1. Is the graph directed?
↓
2. Is it a DAG?
↓
3. What does the DP state mean?
↓
4. Does $dp[u]$ depend on parent or child?
↓
5. Choose topo or reverse topo.
↓
6. What operation combines information?
↓
7. Do I need reachability initialization?
↓
8. Do I need parent/child for reconstruction?

Then identify the operation:

MAX	→ longest/best
MIN	→ shortest/earliest
SUM	→ number of ways
CANDIDATE	→ best node/index
SET	→ all reachable/ancestor information

Core Master Template

```
// 1. Build graph  
// 2. Compute indegree  
// 3. Get topological order  
// 4. Define DP state  
// 5. Initialize source/target  
// 6. Process in the correct dependency order  
// 7. Apply edge transition  
// 8. Reconstruct path if required
```

The main progression learned in this chat was:

```
Basic Topo Sort  
↓
```

Topo + Cycle Detection



Topo + Levels



Topo + Forward DP



Topo + Reverse DP



Topo + Path Reconstruction



Topo + State Propagation



Topo + Greedy